

SCHOOL OF COMPUTING, AMARAVATI – 522503.

TITLE OF THE PROJECT

Submitted by

Student name 1 [Reg no.]

Student name2 [Reg no.]

Student name3 [Reg no.]

Student name4 [Reg no.]

Under the guidance of

Guide Name

accomplishment of the laboratory work for the subject

22AIE441 Dynamics and Control of Robotics.

ABSTRACT

Railway safety is an important concern in transportation, as obstacles on railway tracks can lead to delays and potentially dangerous situations. This project focuses on developing an autonomous railway robot that can detect and remove obstacles from railway tracks using a simulation-based approach. The system is designed to combine robot movement, obstacle detection, robotic arm manipulation, train safety monitoring, and controller analysis in a single environment.The robot identifies obstacles within its detection range and classifies them according to predefined criteria to determine the appropriate handling action. When an obstacle is considered removable, the robotic arm performs a sequence of operations, including reaching, grasping, lifting, moving the obstacle away from the track, and returning to its initial position. The system also monitors train distance and estimates the time available for safe intervention. Warning and emergency-stop mechanisms are incorporated to support safer operation during obstacle removal.

In addition, the project includes machine-learning-based analysis to examine simulated system states and recommend suitable robot actions. A controller tuning module evaluates tracking errors to support adjustments to proportional-derivative (PD) controller gains. The simulation provides visual feedback and event logging, making it easier to observe robot behaviour and analyse system performance.Overall, this project demonstrates how robotics, mathematical control, obstacle handling, and safety monitoring can be integrated into a simulated railway environment. It provides a foundation for further improvements in autonomous railway maintenance and the development of more reliable obstacle-removal systems.

CONTENTS

CHAPTER	TITLE	PAGE NOs
	ABSTRACT	
	LIST OF TABLES	
	LIST OF FIGURES	
	LIST OF ABBREVIATIONS	
1	**INTRODUCTION**	
1.1	Introduction (Background)	
1.2	Problem Statement	
1.3	Objectives	
1.4	Contributions	
2	**RELATED WORK**	
2.1	Comprehensive Survey	
2.2	Summary and Findings of 2.1 in Tabular Format	
2.3	Challenges	
2.4	Motivation of the Project Work	
3	**MATERIALS AND METHODS**	
3.1	Mathematical Modelling/Datasets Description	
3.2	Method(s) of Mathematical Modelling/Data Processing	
3.3	Programming Tools Details	
4	**METHODOLOGY**	
3.2	Controller/Algorithm Design (Mathematical Equations with Description)	

3.3	Controller/Algorithm Design Steps	
3.4	Project Architecture	
3.5	Stability/Algorithm Analysis	
3.6	Project Complete Flowchart	
4	**RESULTS AND DISCUSSION**	
4.1	Simulation Setup	
4.2	Qualitative Results	
4.3	Quantitative Results	
4.4	Discussion	
5	**CONCLUSION AND FUTURE DIRECTIONS**	
5.1	Conclusion	
5.2	Future Directions	
6	**REFERENCES**	

1. INTRODUCTION:-

1.1Introduction (Background)

Railway transportation plays an important role in connecting cities and transporting people and goods. Maintaining railway tracks in a safe condition is essential for the smooth operation of trains. However, obstacles such as stones, fallen objects, and other unwanted materials on railway tracks may create safety risks and interrupt normal train movement. Therefore, developing systems that can identify and handle such obstacles is an important application of robotics and automation.

Robotics offers a potential solution by combining obstacle detection, controlled movement, and robotic manipulation. An autonomous railway robot can be designed to inspect the track area, identify obstacles, and move removable objects away from the track. However, performing these tasks requires more than a mechanical design. The robot must also use suitable mathematical models, motion controllers, and safety mechanisms to coordinate its actions.

This project develops a simulation-based railway obstacle-removal robot that integrates obstacle detection, obstacle classification, robotic arm manipulation, and train safety monitoring. The simulation models the robot's operations and uses control techniques to support movement and manipulation. It also incorporates machine-learning-based analysis and controller tuning to study system behaviour and assist in selecting suitable actions.

The project is implemented in a simulated environment, allowing the robot's behaviour and obstacle-removal process to be observed without the risks associated with physical railway testing. The main focus is to demonstrate how robotics, control systems, and safety monitoring can work together to support autonomous railway maintenance. The simulation also provides a foundation for future improvements in obstacle handling, control performance, and real-world applicability.

1.2 Problem Statement:-

Obstacles on railway tracks can affect train safety and normal operations. Manually identifying and removing these obstacles may take time and require human effort. Therefore, a system is needed to detect obstacles and remove them while considering train safety. This project addresses the problem by simulating a railway robot that identifies obstacles, removes suitable objects using a robotic arm, and monitors train movement. The aim is to study how robot control and safety mechanisms can support the obstacle-removal process in a simulated environment.

1.3 Objectives:-

The main objectives of this project are:

- To develop a simulated railway robot for detecting and removing obstacles from railway tracks.
- To classify obstacles and determine suitable actions for handling them.
- To design robotic arm movements for grasping and removing suitable obstacles.
- To monitor train distance and include warning and emergency-stop mechanisms for safety.
- To study robot movement and tracking performance using controller analysis.
- To use machine-learning-based analysis to examine simulated states and recommend suitable actions.
- To evaluate the robot's behaviour and obstacle-removal process in the simulation environment.

1.4 Contributions:-

- Developed a simulation-based railway robot for obstacle detection and removal.
- Integrated obstacle classification with robotic arm movements to handle suitable obstacles.
- Included train-distance monitoring, warning alerts, and emergency-stop mechanisms to support safety.
- Incorporated machine-learning-based analysis to study simulated system states and recommend actions.
- Added controller analysis to evaluate robot tracking performance and support controller tuning.
- Combined these features in one simulation environment to study the coordination of robot movement, obstacle removal, and train safety.

2. RELATED WORK:-

2.1 Comprehensive Survey:-

Existing research on railway safety has explored obstacle detection, automated monitoring, and robotic systems. Sensors and image-processing techniques can help identify objects on railway tracks, while robotic manipulators can be used to pick up and move suitable obstacles. Control systems play an important role in coordinating robot movement and robotic arm operations. Safety monitoring is also necessary to ensure that obstacle removal is considered alongside train movement and the time available for intervention. However, detecting an obstacle alone is not enough. A complete system must also classify the obstacle, select an appropriate action, control the removal process, and respond to potential safety risks. This project brings these aspects together in a simulation by combining obstacle classification, robotic arm manipulation, train-distance monitoring, warning and emergency-stop mechanisms, and controller analysis. Machine-learning-based analysis is also included to examine simulated system states and recommend suitable actions.

2.3 Challenges:-

- **Obstacle detection and classification:** Identifying obstacles and selecting suitable actions for handling them.
- **Robotic arm control:** Moving the arm accurately to grasp and remove obstacles from the track.
- **Train safety:** Monitoring the train's distance and providing warnings or stopping the system when necessary.
- **Coordinated operation:** Ensuring that obstacle detection, robot movement, and arm manipulation work together properly.
- **Controller performance:** Maintaining accurate robot movement and reducing tracking errors.
- **Action selection:** Using machine-learning-based analysis to examine simulated states and recommend suitable actions.
- **Simulation limitations:** A simulated environment cannot fully represent every real-world railway condition, so further testing would be needed before practical deployment.

Motivation of the Project Work :-

The main motivation for this project is to improve railway safety by developing a robot that can detect and remove obstacles from railway tracks. Manual obstacle removal may require time and human effort, so an automated approach could help support railway maintenance.

This project combines obstacle classification, robotic arm manipulation, train safety monitoring, and controller analysis in a simulation environment. It also uses machine-learning-based analysis to study simulated states and recommend suitable actions. The project aims to understand how these techniques can work together to make obstacle removal more systematic and safer, while providing a foundation for future improvements and real-world testing.

3. MATERIALS AND METHODS:-

3.1 Mathematical Modelling/Datasets Description:-

1. Robot Motion Model

The robot updates its position according to its speed and the simulation time step.

Change in robot position along the track:

$\Delta x = v \times \Delta t$

Therefore:

$x(new) = x(current) + v \times \Delta t$

where x is the robot's position, v is its speed, and Δt is the simulation time step.

The robot's speed is adjusted gradually according to its target speed and acceleration.

2. Robotic Arm Forward Kinematics

The manipulator has four joints: base rotation, shoulder, elbow, and wrist. Forward kinematics calculates the gripper's position from the joint angles and link lengths.

For a simplified planar arm, the end-effector position can be represented as:

$x = L_1 \cos(q_2) + L_2 \cos(q_2 + q_3) + L_3 \cos(q_2 + q_3 + q_4)$

$z = L_1 \sin(q_2) + L_2 \sin(q_2 + q_3) + L_3 \sin(q_2 + q_3 + q_4)$

where L_1, L_2, and L_3 represent link lengths, and q_2, q_3, and q_4 represent joint angles. The complete simulation also includes base rotation and uses transformation matrices to calculate the gripper's world position.

3. PD Controller

The Proportional-Derivative (PD) controller reduces the difference between the desired and actual joint positions.

Position error:

$e = q(desired) - q(actual)$

Velocity error:

$\dot{e} = \dot{q}(desired) - \dot{q}(actual)$

PD control law with gravity compensation:

$$\tau = Kp \times e + Kd \times \dot{e} + G(q)$$

where τ is the control torque, Kp is the proportional gain, Kd is the derivative gain, and G(q) represents the gravity-compensation term. The controller helps the arm follow its desired movement while reducing tracking error.

4. Robot Arm Trajectory Generation

The project uses a fifth-order polynomial to produce smooth joint movement.

Normalized time:

$$s = t / T$$

Quintic trajectory function:

$$f(s) = 10s^3 - 15s^4 + 6s^5$$

Joint position:

$$q(t) = q(start) + [q(target) - q(start)] \times f(s)$$

where t is elapsed time and T is the trajectory duration. This method provides zero initial and final velocity and acceleration, helping avoid abrupt arm movements.

5. Obstacle Distance Calculation

The forward distance between the robot and an obstacle is calculated as:

$$d = x(obstacle) - x(robot)$$

This distance helps the controller decide when to slow down, approach an obstacle, or begin the removal process.

6. Obstacle-Risk Assessment

The project calculates a risk score using obstacle type, mass, lateral position, and size.

Mass modifier:

$$M = 15 \times \min(mass / 200, 1)$$

Position modifier:

$$P = 10 \times (1 - lateralOffset)$$

Size modifier:

$S = 10 \times \min(\text{size} / 0.65, 1)$

Total risk score:

$\text{Risk} = \min(\text{TypeBase} + M + P + S, 100)$

The risk level is classified as LOW for scores up to 30, MEDIUM for scores above 30 and up to 60, and HIGH for scores above 60. The score supports decisions about whether to skip an obstacle or prioritize its removal.

7. Train Time-to-Collision Estimation

The estimated time available before a train reaches an obstacle is calculated using:

$\text{TTC} = \text{distance} / \text{speed}$

where distance is the separation between the train and the relevant hazard, and speed is the train's positive approach speed. If speed is zero or negative, the model returns an infinite time estimate.

8. Machine-Learning-Based Action Selection

The project uses a k-nearest neighbours (k-NN) classifier to recommend an action based on ten features, including normalized obstacle distance, mass, obstacle type, four arm joint angles, gripper position error, robot speed, and train distance.

Euclidean distance between the current feature vector and a reference sample is:

$D = \sqrt{[\Sigma\ (\text{current feature} - \text{reference feature})^2]}$

The classifier identifies the five nearest reference samples and selects the most frequent action label. Its confidence score is based on the relative inverse-distance weights of the neighbours supporting that label.

9. Gradient-Descent Controller-Gain Tuning

The controller analysis evaluates the difference between desired and actual arm joint angles.

Tracking error:

$e = q(\text{desired}) - q(\text{actual})$

Root-mean-square tracking error:

$\text{RMSE} = \sqrt{[\text{mean}(e^2)]}$

The loss function used for gain tuning is:

$$L = \text{mean}(e^2) + \lambda \times [\Sigma(Kp^2) + \Sigma(Kd^2)]$$

The controller gains are updated using gradient descent:

$$Kp(new) = Kp(old) - \eta \times \text{gradient}(Kp)$$

$$Kd(new) = Kd(old) - \eta \times \text{gradient}(Kd)$$

where η is the learning rate and λ is the regularization coefficient. The updated gains are restricted to specified limits.

10. Dataset Description

The project primarily uses simulation-generated logs and a small, hand-crafted reference dataset for the k-NN classifier. The simulation records information such as robot position and speed, arm joint angles, desired joint positions, tracking errors, obstacle properties, and system states. These data support analysis of robot behaviour, controller performance, and obstacle-handling decisions.

Overall, these mathematical models connect robot movement, arm manipulation, safety decisions, and machine-learning-based analysis within the railway obstacle-removal simulation.

3.2 Method(s) of Mathematical Modelling/Data Processing:-

The project uses mathematical modelling, numerical calculations, control algorithms, and simulation-based data processing to coordinate robot movement and obstacle removal. MATLAB is used to implement these methods and visualize the simulation results.

1. Robot Dynamics Modelling:-

The robotic arm is modelled using rigid-body dynamics. The mass matrix, Coriolis and centripetal effects, and gravity are considered to calculate the required joint torques.

$$M(q) \times \ddot{q} + C(q, \dot{q}) \times \dot{q} + G(q) = \tau$$

Here, $M(q)$ represents the mass and inertia matrix, $C(q, \dot{q})$ represents velocity-related effects, $G(q)$ represents gravity effects, and τ represents the joint torque vector. This model is used to analyse the arm's dynamic behaviour.

2. PD Control and Trajectory Tracking:-

A Proportional-Derivative (PD) controller is used to reduce the difference between desired and actual joint positions. Gravity compensation is included in the torque calculation.

Joint position error:

$e = q(\text{desired}) - q(\text{actual})$

Control torque:

$\tau = Kp \times e + Kd \times \dot{e} + G(q)$

The desired arm movement is generated using smooth trajectories. The controller analysis evaluates tracking errors and torque requirements to study the movement accuracy of the robotic arm.

3. Forward Kinematics and Arm Movement:-

Forward kinematics calculates the gripper position from the joint angles and arm-link dimensions. Transformation matrices are used to determine the gripper's position and orientation relative to the robot and the railway environment. This information helps the manipulator approach, grasp, and move suitable obstacles.

4. Obstacle Detection and Classification:-

The obstacle scanner calculates the distance between the robot and nearby obstacles. The obstacles are then classified according to their properties, such as type and mass. Based on the classification and risk assessment, the system selects an appropriate action, such as approaching an obstacle, using the robotic arm, pushing a heavy obstacle, or stopping for safety.

5. Train-Safety and Risk Assessment:-

The system monitors the distance between the train and the relevant hazard. The estimated time to collision is calculated as:

TTC = Distance to hazard / Train speed

The risk assessment also considers obstacle properties and position. These calculations support safety decisions, including warnings and emergency stopping when the situation requires it.

6. Machine-Learning-Based Data Processing:-

The machine-learning module extracts ten features from the simulation state, including normalized obstacle distance, obstacle mass, encoded obstacle type, four arm joint angles, gripper position error, robot speed, and train distance.

The k-nearest neighbours (k-NN) method compares the current feature vector with a labelled reference dataset. Euclidean distance is calculated as:

$D = \sqrt{[\Sigma\ (\text{Current feature} - \text{Reference feature})^2]}$

The five nearest samples are selected, and majority voting determines the recommended action. A distance-weighted confidence score is also calculated. The predicted action is compared with the robot's current action to analyse whether the two agree.

7. Controller-Gain Optimization:-

The controller-analysis module uses gradient descent to suggest proportional and derivative gain adjustments based on tracking error. The loss function is:

L = Mean squared tracking error + λ × (Sum of squared Kp gains + Sum of squared Kd gains)

The general update rule is:

New gain = Old gain − Learning rate × Gradient of loss

The updated gains are constrained within specified limits. This process is intended to reduce tracking error while avoiding unnecessarily large controller gains.

8. Simulation Data Logging and Visualization:-

The simulation records relevant information about robot movement, obstacle handling, train safety, and system states. MATLAB plots are used to examine joint tracking errors, control torques, phase-plane behaviour, and manipulator movement. These results help evaluate the performance of the control system and identify areas for improvement.

Overall, these methods combine mathematical modelling, feedback control, kinematic calculations, safety assessment, machine-learning-based action analysis, and controller optimization to study the railway robot's behaviour in a simulated environment.

3.3 Programming Tools Details:-

MATLAB is used to develop the railway robot simulation. It helps us perform mathematical calculations, control the robot's movement, and observe how the robot behaves while removing obstacles from the railway track.

The main tools and functions used in the project are:

- **MATLAB:** Used to implement the robot control methods, mathematical calculations, obstacle detection, and safety monitoring.
- **3D Graphics:** MATLAB graphics functions are used to create and display the railway track, train, robot, robotic arm, and obstacles.
- **Robot Control:** Control algorithms are used to guide the robot's movement and control the robotic arm. The controller analysis also helps us study tracking errors and joint torques.
- **Obstacle Detection and Safety:** The program checks obstacle distance, estimates the time available before a train reaches a hazard, and supports decisions related to obstacle removal and emergency stopping.

- **Machine Learning:** The k-nearest neighbours (k-NN) method analyses simulation data and recommends suitable actions based on the reference data available in the program.
- **Data Analysis and Visualization:** MATLAB plots and simulation logs help us understand the robot's movement, controller performance, and obstacle-handling process.

4. METHODOLOGY:-

4.1 Controller/ Algorithm Design (Mathematical Equations with Description):-

Robot Dynamics Model:-

The dynamics of the robotic arm are represented by the following equation:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$$

Where:

$M(q)$ – Mass and inertia matrix of the robotic arm.

q – Joint angles.

$\dot{q}$ – Joint velocities.

$\ddot{q}$ – Joint accelerations.

$C(q, \dot{q})$ – Matrix representing Coriolis and centrifugal effects.

$G(q)$ – Gravitational torque vector.

τ – Applied joint torques.

This equation describes the relationship between the arm's movement, physical properties, gravity, and applied torques.

PD Controller with Gravity Compensation:-

The controller calculates the required joint torques using the following equation:

$$\tau = Kp(qd - q) + Kd(\dot{q}d - \dot{q}) + G(q)$$

The position error is:

$$e = qd - q$$

The velocity error is:

$\dot{e} = \dot{q}d - \dot{q}$

Where:

Kp – Proportional controller gain.

Kd – Derivative controller gain.

qd – Desired joint angles.

q – Actual joint angles.

$\dot{q}d$ – Desired joint velocities.

$\dot{q}$ – Actual joint velocities.

G(q) – Gravity compensation term.

The proportional term reduces position error, while the derivative term helps control velocity error and unwanted oscillations. Gravity compensation accounts for the effect of gravity on the robotic arm.

Forward Kinematics:-

Forward kinematics determines the end-effector position from the joint angles. For a simplified two-link planar robotic arm, the equations are:

$x = L1 \cos(q1) + L2 \cos(q1 + q2)$

$y = L1 \sin(q1) + L2 \sin(q1 + q2)$

Where:

x, y – End-effector position.

L1, L2 – Lengths of the two arm links.

q1, q2 – Joint angles.

These equations illustrate how joint rotations determine the end-effector position.

Obstacle Detection and Classification

The robot scans the railway track to detect obstacles and estimate their distance.

d_o = Distance between the robot and the obstacle

Where:

d_o – Distance from the robot to the detected obstacle.

Based on the obstacle's properties and risk level, the robot selects an appropriate action. It may continue moving, skip a low-risk obstacle, use the robotic arm to remove a suitable object, or use the front plow to push a heavy obstacle aside.

Train Safety Monitoring:-

The time-to-collision (TTC) equation estimates the time available before a moving train reaches a potential hazard.

$TTC = d / v$, where $v > 0$

Where:

TTC – Estimated time before the train reaches the relevant point.

d – Distance to the train.

v – Train speed.

This calculation assumes constant speed. The safety module uses the estimate and train status to support warning and emergency-stop decisions.

Machine-Learning-Based Action Prediction

The project uses the k-nearest neighbours (k-NN) algorithm to recommend actions based on the current simulated state. The Euclidean distance between the current state and a reference sample is calculated as:

$D_i = \sqrt{[\Sigma_{j=1}^{n} (x_j - r_{ij})^2]}$

Where:

D_i – Distance between the current state and reference sample i.

x_j – Feature j of the current simulated state.

r_{ij} – Corresponding feature j of reference sample i.

n – Number of features used in the comparison.

The algorithm selects the five nearest reference samples and uses distance-weighted voting to predict an action. The recommendation is analysed alongside the action selected by the robot's existing logic.

Controller Gain Optimization:-

A gradient-descent-based module analyses tracking errors and suggests adjustments to the PD controller gains.

The tracking error is:

$e_i = qd,_i - q_i$

The loss function is:

$L = (1/N) \Sigma_{i=1}^{N} \|e_i\|^2 + \lambda(\|Kp\|^2 + \|Kd\|^2)$

The gain update equations are:

$Kp(new) = Kp - \eta \nabla Kp\ L$

$Kd(new) = Kd - \eta \nabla Kd\ L$

Where:

L – Loss function measuring tracking error and gain regularization.

N – Number of samples used for evaluation.

e_i – Joint tracking error for sample i.

λ – Regularization parameter.

η – Learning rate controlling the size of each update.

$\nabla Kp\ L$ – Gradient of the loss with respect to Kp.

$\nabla Kd\ L$ – Gradient of the loss with respect to Kd.

The optimizer limits the updated gains to predefined ranges. It supports controller performance analysis and suggests gain adjustments rather than automatically replacing the gains used by the main simulation.

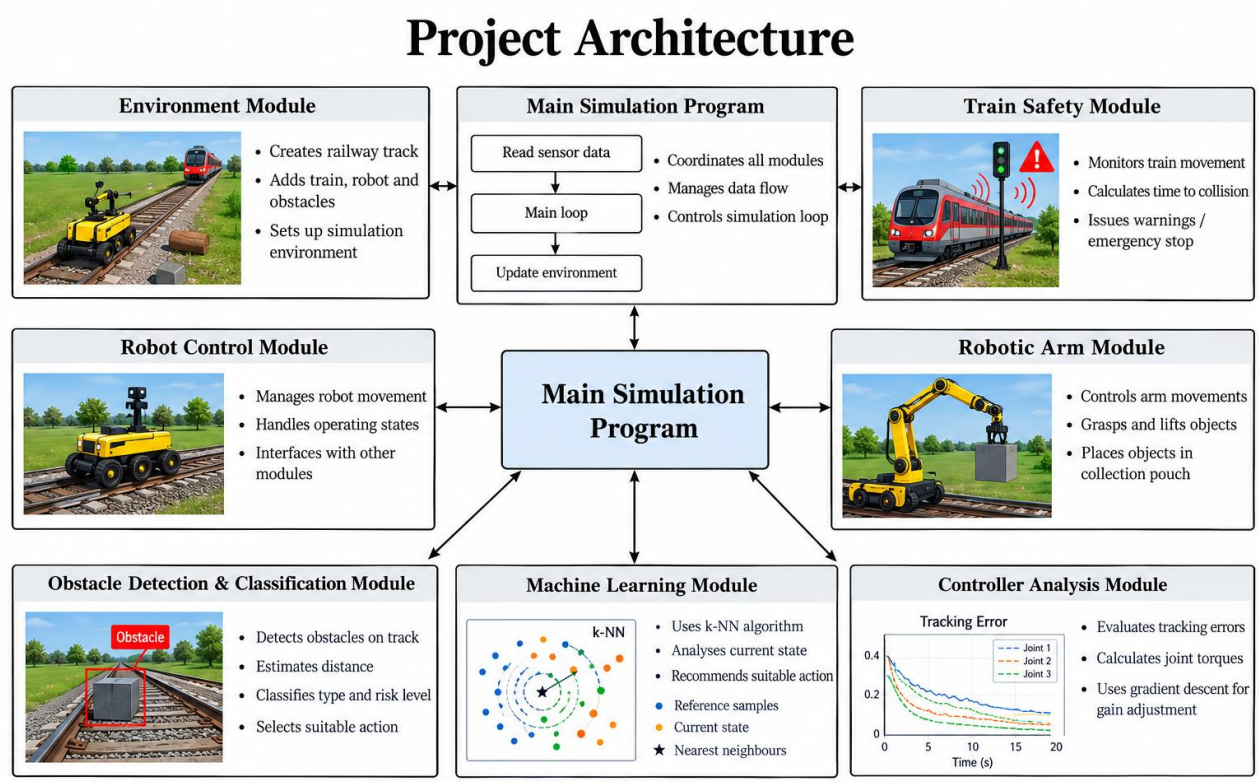
Overall, these mathematical models and algorithms support robotic arm control, obstacle handling, train safety monitoring, action prediction, and controller performance analysis.

4.2 Controller/ Algorithm Design Steps:-

The railway robot follows a sequence of steps to detect obstacles, select suitable actions, remove obstacles, and maintain safety during operation.

1. **Initialize the Simulation:** Set up the railway track, robot, robotic arm, train, and obstacles. Initialize the required controller parameters and simulation settings.
2. **Start Robot Movement:** The robot begins moving along the railway track while continuously checking the environment.
3. **Detect Obstacles:** The obstacle detection module scans the track and identifies obstacles in the robot's path. The distance to the detected obstacle is evaluated.
4. **Classify the Obstacle:** The system evaluates the obstacle's properties and risk level to determine the appropriate handling method.
5. **Select an Action:** Based on the classification, the robot decides whether to continue moving, skip a low-risk obstacle, remove an object using the robotic arm, or push a heavy obstacle aside using the front plow.
6. **Control the Robotic Arm:** For removable obstacles, the arm moves towards the object, grasps it, lifts it, and places it in the collection pouch before returning to its initial position.
7. **Monitor Train Safety:** The system checks train movement and estimates the time available before the train reaches the relevant point. Warning or emergency-stop actions are applied when required by the safety logic.
8. **Analyse Controller Performance:** The PD controller calculates joint torques for analysis, while tracking errors are monitored. The gradient-descent module evaluates the errors and suggests suitable controller-gain adjustments.
9. **Use Machine-Learning-Based Analysis:** The k-NN algorithm analyses the current simulated state using reference data and recommends a suitable action. Its recommendation can be compared with the action selected by the existing robot logic.
10. **Verify Obstacle Removal:** After the handling operation, the system checks whether the obstacle has been cleared from the track.
11. **Resume Robot Movement:** If the track is safe and no emergency condition is active, the robot continues patrolling and checks for additional obstacles.
12. **Complete the Simulation:** After the required obstacle-handling tasks are completed, the robot returns to its home position and the simulation records the final events and performance information

4.3 Project Architecture:-



4.4 **Stability/Algorithm Analysis:-**

The stability and performance of the railway robot are studied by analysing robot movement, joint tracking errors, calculated torques, and obstacle-handling behaviour. The PD controller uses proportional and derivative gains to reduce the difference between the desired and actual joint positions.

The joint tracking error is calculated as:

$e(t) = qd(t) - q(t)$

Where:

$e(t)$ – Joint tracking error.

$qd(t)$ – Desired joint position.

$q(t)$ – Actual joint position.

A smaller tracking error indicates that the actual joint position is closer to the desired position. The controller also considers joint velocity errors to improve movement and reduce unwanted oscillations.

The calculated joint torques are limited to predefined maximum values to avoid excessive torque commands. The gradient-descent-based module analyses tracking errors and suggests adjustments to the PD controller gains to improve tracking performance.The algorithm's performance is also evaluated by observing obstacle detection, obstacle removal, robot movement, and train safety responses. The simulation checks whether the robot selects suitable actions and resumes movement after an obstacle-handling operation when it is safe to do so.The project provides a simulation-based evaluation of controller behaviour and algorithm performance. However, formal mathematical proof of closed-loop stability is not established by tracking-error analysis alone. Further testing under different operating conditions would be required to verify stability and reliability for real-world railway applications.

5. RESULTS AND DISCUSSION:-

5.1 Simulation Setup:-

The railway robot simulation is developed in MATLAB to study obstacle detection, obstacle removal, robot control, and train safety. The environment includes a railway track, a train, a mobile robot, a robotic arm, and obstacles placed along the track.

The robot moves along the track and scans for obstacles. When an obstacle is detected, the system evaluates its properties and selects a suitable action. The robotic arm handles removable objects, while the front plow is used to push aside heavy obstacles. A collection pouch is provided to store the removed objects. The simulation also includes train safety monitoring, a PD controller for controller analysis, and machine-learning-based action recommendations. Tracking errors, calculated joint torques, and simulation events are monitored to study system behaviour. The simulation provides a visual environment for observing the robot's movement, obstacle-handling process, and safety responses under the defined operating conditions.

5.2 Qualitative Results:-

- **Robot Movement:** The robot moves along the railway track and checks for obstacles in its path.
- **Obstacle Detection:** When an obstacle is detected, the system checks its properties and decides how to handle it.
- **Obstacle Removal:** The robotic arm handles suitable obstacles, while the front plow is used to push heavy obstacles aside.
- **Collection of Obstacles:** The collection pouch provides space to store objects removed by the robotic arm.
- **Train Safety:** The system monitors train movement and supports warning and emergency-stop actions when required.
- **Controller Performance:** The controller analysis helps observe joint tracking errors and calculated torques during operation.

- **Machine-Learning Analysis:** The k-NN algorithm analyses simulated data and recommends suitable actions for the current state.
- **Simulation Visualization:** The simulation displays robot movement and obstacle-handling activities, making it easier to understand how the different modules work together.

Overall, the simulation demonstrates how robot movement, obstacle handling, controller analysis, and safety monitoring can work together in a railway environment.

5.3 Quantitative Results:-

- **Obstacle Detection:** The simulation allows us to observe and record the number of obstacles detected during robot operation.
- **Obstacle Removal:** The number of obstacles successfully removed or pushed aside can be used to evaluate the robot's performance.
- **Robot Movement:** The robot's position and movement can be monitored to understand its progress along the railway track.
- **Controller Performance:** Joint tracking errors can be analysed to check how closely the robotic arm follows its desired movements.
- **Joint Torque Analysis:** The calculated joint torques help us study the effort required for the robotic arm's movements.
- **Train Safety Monitoring:** The distance between the train and the obstacle can be monitored to support timely warning and emergency-stop decisions.
- **Machine-Learning Analysis:** The k-NN module provides action recommendations based on the simulated state data.
- **Overall Performance:** Simulation logs and recorded measurements can be used to evaluate obstacle handling, robot movement, and safety responses.

The quantitative results help evaluate the robot using measurable parameters. Actual numerical values should be taken from the simulation logs and plots; no specific performance values are included here because verified measurements have not been provided.

5.4 Discussion:-

- **Obstacle Handling:** The simulation demonstrates how the robot identifies obstacles and selects suitable actions based on their properties.
- **Robotic Arm Operation:** The robotic arm is used to handle removable obstacles, while the front plow helps push heavier obstacles aside.
- **Safety Monitoring:** Monitoring the train's distance helps the system make decisions about warnings and emergency stops during obstacle-handling operations.
- **Controller Performance:** Analysing tracking errors and calculated joint torques helps us understand the robotic arm's behaviour and identify areas for improvement.
- **Machine-Learning Support:** The k-NN algorithm analyses simulated data and recommends actions based on the current system state.
- **System Integration:** Combining robot movement, obstacle handling, controller analysis, and train safety monitoring allows us to study how these components work together.

- **Limitations:** The results are based on simulation. Real railway conditions, hardware limitations, and environmental disturbances may affect performance differently.

Key Findings: The project demonstrates how robotics, control techniques, and machine-learning-based analysis can be combined to support railway obstacle removal. The simulation provides a useful foundation for evaluating obstacle-handling performance, improving controller behaviour, and strengthening safety mechanisms before further real-world testing.

6. CONCLUSION AND FUTURE DIRECTIONS:-

6.1 Conclusion:-

This project demonstrates a railway robot simulation designed to detect and remove obstacles from railway tracks. The robot identifies obstacles, selects suitable actions, and uses a robotic arm or front plow to handle them. The system also monitors train movement to support safety-related decisions. Controller analysis and machine-learning-based action recommendations help us study the robot's behaviour during the simulation.

The project helped us understand how robotics, control techniques, and machine learning can be combined to address railway obstacle removal. Although the simulation demonstrates the basic working of the system, further improvements and testing are needed before it can be considered for real-world use.

6.2 Future Directions:-

Future research can extend the existing work on railway service robots by focusing on autonomous obstacle detection and removal on railway tracks. The robot's kinematic and dynamic models can be integrated with suitable control algorithms to improve robotic arm movement during obstacle handling. Further research can explore intelligent obstacle classification and action selection based on obstacle properties and train movement. The system can also be extended to more realistic simulation environments to evaluate its performance under different operating conditions. Finally, the proposed approach can be validated through physical robot experiments to investigate its feasibility for practical railway maintenance.

REFERENCES:-

[1] G. D'Amico, M. Marinoni, F. Nesti, G. Rossolini, G. C. Buttazzo, S. Sabina, and G. Lauro, "TrainSim: A Railway Simulation Framework for LiDAR and Camera Dataset Generation," IEEE Transactions on Intelligent Transportation Systems, vol. 24, no. 12, pp. 15006–15017, 2023.

[2] H. Liu, M. Rahman, M. Rahimi, A. Starr, I. Durazo-Cardenas, C. Ruiz-Carcel, A. Ompusunggu, A. Hall, and R. Anderson, "An Autonomous Rail-Road Amphibious Robotic System for Railway Maintenance Using Sensor Fusion and Mobile Manipulator," Computers & Electrical Engineering, vol. 110, p. 108874, 2023.

[3] "Research and Application of the Obstacle Avoidance System for High-Speed Railway Tunnel Lining Inspection Train Based on Integrated 3D LiDAR and 2D Camera Machine Vision Technology," Applied Sciences, vol. 13, no. 13, p. 7689, 2023.